

Assignment - 1

1

Page No.:

Date: / /

- Q-1 Define one-to-one, onto and bijection function.
- check whether the function $f: \mathbb{R}_+ \rightarrow \mathbb{R}$, $f(x) = x^2$ is one to one and onto.
 - Explain equivalence relation with example.
 - Explain reflexivity, symmetry and transitivity properties of relations.

★ One-to-one function: A function for which every element of the range of the function corresponds to exactly one element of the domain is known as one-to-one or injective or injection.

★ Onto function: If the range of function and codomain of function are equal or every element of the codomain is actually one of the values of the function, then function is said to be onto or surjective or surjection.

★ Bijection function: If function is both one-to-one and onto then function is called Bijection function.

① check whether the function $f: \mathbb{R}_+ \rightarrow \mathbb{R}$, $f(x) = x^2$ is one to one and onto.

→ The range and codomain of $f(x) = x^2$ are not equal so, function f is not onto function.

→ The function is one to one because elements of B are connected with single element of A .

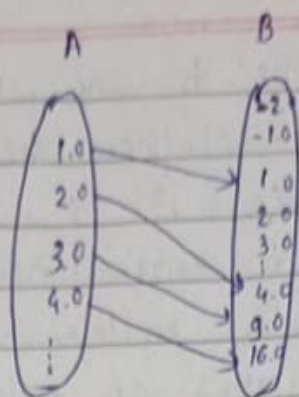
$$RZY \neq YZ$$

$$f(1) = 1$$

$$f(2) = 4$$

$$f(3) = 9$$

$$f(4) = 16$$



(b) Explain equivalence Relations with examples.

→ R is an equivalence relation on A , if R is Reflexive, symmetric and transitive

example: $A = \{a, b\}$, $R = \{(a, a), (b, b), (a, b), (b, a)\}$

→ Reflexive, for every $x \in A$, $x R x$

→ $a \in A$, $a R a$

→ $b \in A$, $b R b$

⇒ $\{(a, a), (b, b)\}$

→ Symmetric: for every a, b in A ,

if $a R b$, $b R a$ is true

⇒ $\{(a, b), (b, a)\}$

→ Transitive ⇒ For every $a, b \in A$,

$\{(a, a), (a, b), (a, b),$

$(b, b), (b, a), (b, a),$

$(a, b), (b, b), (a, b),$

$(a, b), (b, a), (a, a),$

$(b, a), (a, b), (b, b)$

$(b, a), (a, a), (b, a)\}$

∴ Above relation is equivalence Relation because it is Reflexive, symmetric and transitive.

(c) Explain Reflexive, symmetric, and transitivity properties of relation

→ R is a Relation on set A ,

$R \subseteq A \times A$, where $(x, y) \in R$ to indicate x is related to y via Relation R .

- ① Reflexivity: R is reflexive if for every $x \in R$, xRx .
- ② Symmetry: R is symmetric if for every x and y in A , if xRy , then yRx .
- ③ Transitivity: R is transitive if for every x, y, z in A if xRy , yRz then xRz .

Q-2 State the principle of mathematical induction and prove by mathematical Induction for the following.

- (a) prove that all positive integers n ,
 $1 + 2 + 3 + \dots + n = n(n+1)/2$

→ Step-1: Basic Step, we must show that $P(1)$ is true.

$$P(1) = 1 \text{ (L.H.S)}$$

$$P(1) = 1(1+1)/2 = 1, \text{ And this is obviously true.}$$

→ Step-2: Induction Hypothesis

$$k \geq 1 \text{ and } 1 + 2 + 3 + \dots + k = k(k+1)/2$$

→ Step-3: Proof of Induction

$$P(k+1) = 1 + 2 + 3 + \dots + k + (k+1)$$

$$= k(k+1)/2 + (k+1)$$

$$= (k(k+1) + (k+1)^2) / 2$$

$$= ((k+1)(k+2)) / 2$$

$$= (k+1)(k+1+1) / 2$$

Hence proved

(b) Prove that $\sqrt{2}$ is Irrational.

Definition $\rightarrow$ A real number is rational if there are two integers m and n so that $x = m/n$.

Proof:-

- $\rightarrow$ Suppose for sake of contradiction that $\sqrt{2}$ is rational.
- $\rightarrow$ Then there exists some integers m' and n' such that $\sqrt{2} = m'/n'$.
- $\rightarrow$ By dividing both m' and n' by all the factors that are common to both, we obtain $\sqrt{2} = m/n$, for some integers m and n having no common factors.
- $\rightarrow$ Since $\sqrt{2} = m/n$, $m = n\sqrt{2}$, squaring both sides of this equation, we obtain $m^2 = 2n^2$, and therefore m^2 is even.
- $\rightarrow$ If a and b are odd, then ab is odd, since a conditional statement is logically equivalent to its contrapositive, we may conclude that for any a and b , if ab is not odd, then either a is not odd or b is not odd.
- $\rightarrow$ However, an integer is not odd if and only if it is even, and so for any a and b , if ab is even, a or b is even.
- $\rightarrow$ If we apply this when $a=b=m$, we conclude that since m^2 is even, m must be even, this means that for some k , $m = 2k$, therefore $(2k)^2 = 2n^2$.
- $\rightarrow$ Simplifying and cancelling 2 from both sides, we obtain $2k^2 = n^2$. Therefore n^2 is even and therefore $n = 2j$ for some j . We have shown that m & n are both divisible by 2. This contradicts the previous statement that m & n have no common factor.
- $\therefore$ The assumption that $\sqrt{2}$ is rational therefore leads to contradiction, and conclusion is that $\sqrt{2}$ is irrational.

Q-3 Explain Types of Grammar / Chomsky's Hierarchy

→ Chomsky's Hierarchy of grammars classifies different types of formal grammars based on their production rules and the types of languages they generate.

→ There are four types:

1. Type 0 (Recursively Enumerable or Phrase-Structure Grammars)

- These are the most general grammars, meaning they have the fewest restrictions
- Any rule can replace a string containing non-terminal symbols with other string.
- The languages generated by these grammars are accepted by a Turing Machine (TM).

2. Type-1 (Context-sensitive Grammars)

- Here, rules can replace a nonterminal symbol only if it appears in a certain context.
- The replacement string must be at least as long as the original.
- These languages are accepted by a Linear-Bounded Automaton (LBA), a restricted type of Turing Machine with limited tape size.

3. Type 2 (Context-free Grammars - CFGs)

- Each rule replaces a single non-terminal with some string of terminals and/or non-terminals.
- These grammars generate languages that can be recognized by a Pushdown Automaton (PDA), which has a memory stack.
- CFGs are widely used in programming languages and parsers.

4. Type 3 (Regular Grammars)

- The simplest type, where each rule follows a specific pattern:
- A nonterminal is replaced by a single terminal or a terminal followed by a non-terminal.
- These languages can be recognized by finite Automata (FA), which has no extra memory.
- Regular languages are commonly used for defining patterns, such as those in search engines and regular expressions.

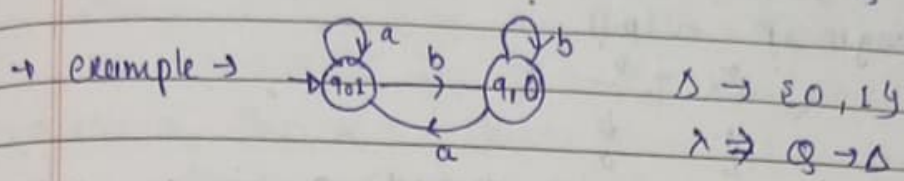
→ Each type of grammar follows stricter rules than the previous one, meaning that Type 3 is a subset of Type 2, which is a subset of Type 1, and so on.

Q-4 Explain Moore Machine and Mealy machine in detail with example.

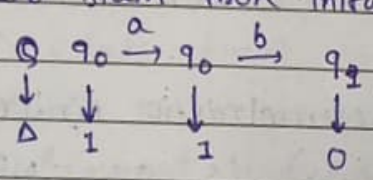
* Moore Machine :

- A Moore machine is a finite state machine where the output depends only on the current state and not on the input. This means that for a given state, the output remains the same until the state changes.
- It consists of states, transitions, inputs and outputs.
- The output is linked to the states, not to the transitions.
- The output only changes when the machine moves to a different state.
- Moore Machine are finite state machines with output value and its output depends only on the present state.

- It can be defined as $(Q, \Sigma, \delta, q_0, \Delta, \lambda)$
 where, $Q \rightarrow$ set of finite state
 $\Sigma \rightarrow$ Input alphabets
 $q_0 \rightarrow$ Initial state
 $\delta \rightarrow$ Transition function.
 $Q \times \Sigma \rightarrow Q$
 $\Delta \rightarrow$ output alphabets.
 $\lambda \rightarrow$ output function. $\lambda \Rightarrow Q \rightarrow \Delta$



- The length of output for Moore Machine is greater than input by 1.
 → In the moore machine shown figure, the output is represented with each input state.
 → First we start from initial state q_0 .



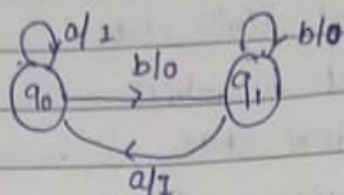
- We get 'ab' using moore's machine.
 → q_0 output alphabet is 1 and q_1 output alphabet is 0.

* Mealy Machine

- A mealy machine is a finite state machine where the output depends on both the current state and the inputs.
 → This means the output can change immediately when the input changes, even if the state remains the same.
 → It also consists of states, transitions, inputs, outputs.
 → The output associated with transitions, not states.
 → The output changes as soon as the input changes making it more responsive.
 → It can be defined as $(Q, \Sigma, \delta, q_0, \Delta, \lambda)$
 Same as moore's machine but one change in output

function is, $\lambda: Q \times \Sigma \rightarrow \Delta$

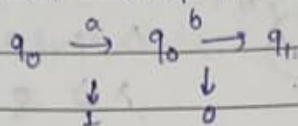
example:-



$\Delta = \{0, 1\}$

$\lambda: Q \times \Sigma \rightarrow \Delta$

- The output is represented with each input symbol for each state represented by 1.
- The length of output for mealy machine is equal to length of output.



- q_0 to q_0 output is 1 and q_0 to q_1 output is 0.

Q-5 Define NFA- ϵ . Explain how to convert NFA- ϵ into NFA with suitable example and NFA to DFA with suitable example.

- An NFA- ϵ (Nondeterministic Finite Automaton with Epsilon (ϵ) transitions) is an NFA that includes a transition labeled with ϵ (epsilon/ ϵ), which allows the automaton to move to new state without consuming any input symbol. This means that an NFA- ϵ can change states spontaneously without reading on an input symbol.

→ formal definition:-

$(Q, \Sigma, \delta, q_0, F)$

$Q \rightarrow$ finite set of states

$\Sigma \rightarrow$ finite set of input symbols

$\delta \rightarrow$ Transition function $\delta: Q \times \Sigma \cup \{\epsilon\} \rightarrow 2^Q$

$q_0 \rightarrow$ Initial state

$F \rightarrow$ set of final states.

- To remove the epsilon move / Null move from epsilon-NFA and to convert it into NFA, we follow the steps:

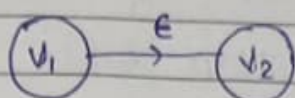
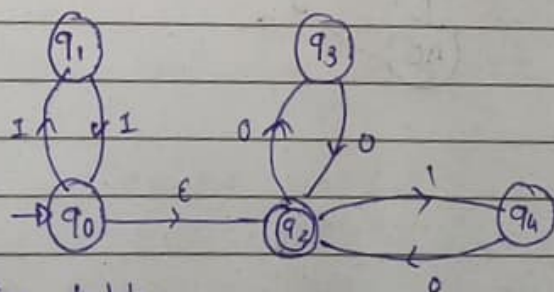


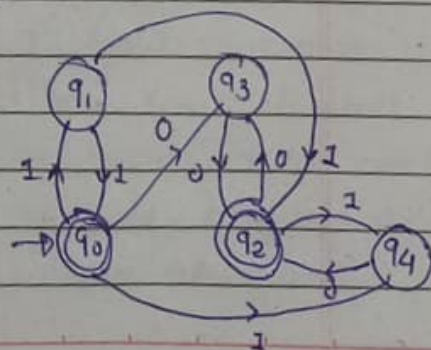
Figure - Vertex V_1 and vertex V_2 having an epsilon move

- Step-1 Identify ϵ -move - find states $V_1 \rightarrow V_2$ connected by an ϵ -transition.
- Step-2 Duplicate transitions: for every transition from V_2 (except ϵ), create the same transition from V_1 and remove $V_1 \rightarrow V_2$ ϵ -move
- Step-3 Update start state: if V_1 is a start state, make V_2 a start state.
- Step-4 Update final state: if V_2 is a final state, make V_2 a final state.
- Step-5 Repeat step-1-4 until all ϵ -moves are removed.
- Example Convert epsilon-NFA to NFA.

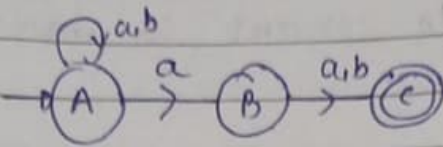


→ Transition table

	0	1
→ q_0	$\{q_3\}$	$\{q_4, q_1\}$
q_1	$\emptyset$	$\{q_0, q_2\}$
q_2	$\{q_3\}$	$\{q_4\}$
q_3	$\{q_2\}$	$\emptyset$
q_4	$\{q_2\}$	$\emptyset$



* NFA to DFA conversion.



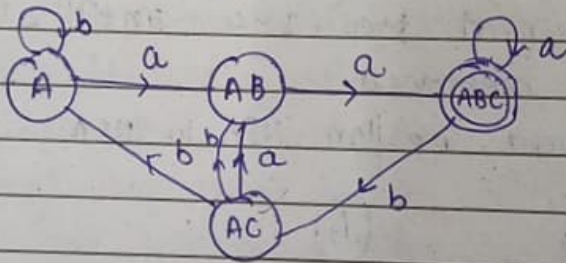
Step-1 Draw transition table.

	a	b
A	$\{A, B\}$	$\{A\}$
B	$\{C\}$	$\{C\}$
C	$\{C\}$	$\{C\}$

Step-2 : Compute transitions of DFA.

	a	b
$\rightarrow [A]$	$[AB]$	$[A]$
$[AB]$	$[ABC]$	$[AC]$
$[AC]^*$	$[AB]$	$[A]$
$[ABC]^*$	$[ABC]$	$[AC]$

Step-3 Draw DFA.



Q-6 state pumping lemma for regular languages.

→ If L is infinite language then there exist some positive integer 'n'. (pumping length) such that any string $w \in L$ has length greater than equals n . that i.e. $|w| \geq n$, then w can be divided into three parts $w = xyz$ and satisfy the following condition,

- (1) for each $i \geq 0$, $xy^iz \in L$
- (2) $|y| > 0$ and
- (3) $|xy| \leq n$.

→ for example :- Infinite language $L = a^n b^{2n}$
 $w \in L$ for $n = 2$

$$w = \underbrace{aa}_x \underbrace{bb}_y \underbrace{bb}_z \quad (\text{divide into three parts})$$

Now pumping the value of y

$$\underbrace{aa}_x \underbrace{bb}_y \underbrace{bb}_y \underbrace{bb}_z \neq z$$

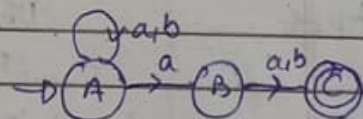
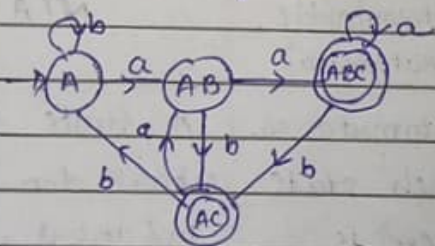
This language is not Regular.

Q-7 Differentiate b/w DFA and NFA with example.

features	DFA (Deterministic Finite Automata)	NFA (Non-DFA)
Definition	A finite automata where for each state and input, there is exactly one transition	A finite automata where for each state and input, there can be multiple transitions (or none)

Determinism	Completely deterministic - given an input, the next state is uniquely determined	Can have multiple possible paths for the same inputs.
Transitions	Each state has only one transition per input symbol.	A state can have multiple transitions for the same ^{input} .
ϵ (epsilon) transitions	No ϵ -transitions (must consume input for transitions)	Can have ϵ -transitions (move b/w states without consuming input)
Complexity	Generally more efficient as it does not require backtracking.	More flexible but can require backtracking or parallel processing.
Implementation	Easier to implement due to deterministic behavior	Can be harder to implement due to multiple possible states at any step.
State explosion	Can require more states to represent a language.	Can be more compact, requiring fewer states.
Expressive power	Equally powerful as an NFA (both recognize the same class of languages: Regular languages)	Equally powerful as a DFA but may have a more compact representation.

Example:



→ This example of that language's ^{string} which second symbol from RHS is a.

Q-8 Construct a Regular Expression over $\Sigma = \{a, b\}$ for set of all strings for the following:

(a) String whose length is at most 2

$$L = \{\epsilon, a, b, aa, bb, ba, ab\}$$

$$RE \Rightarrow (a + b)(a + b)$$

(b) String which starts and ends with different symbols.

$$L = \{ab, abab, \dots\}$$

$$RE \Rightarrow a(a+b)^*b + b(a+b)^*a$$

(c) Strings in which no 2 a's are not together / string should not contain "aa" as substring.

$$L = \{\epsilon, a, b, bb, ab, aba, \dots\}$$

$$RE \Rightarrow (b+ab)^* + (b+ab)^*a$$

$$\Rightarrow (b+ab)^*(\epsilon+a)$$

(d) Strings in which no 2 'a' and 2 'b' are not together / string should not contain 'aa' & 'bb' as substring.

$$L = \{\epsilon, a, b, ab, ba, abab, \dots\}$$

$$RE \Rightarrow (a+b)(ab)^*(a+b)$$

Q-9 Do as directed.

(a) Construct a minimal DFA which accepts set all string over $\Sigma = \{a, b, c\}$ which when interpreted as ^(base 3) ternary no. divisible by 4.

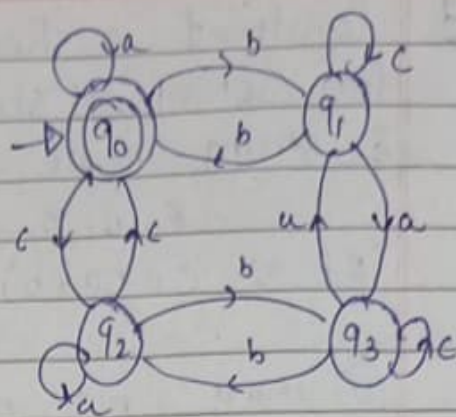
assuming $a = 0, b = 1, c = 2$ ($\because$ interpreted as ternary no.)

$$L = \{\epsilon, a, b, c, \dots\}$$

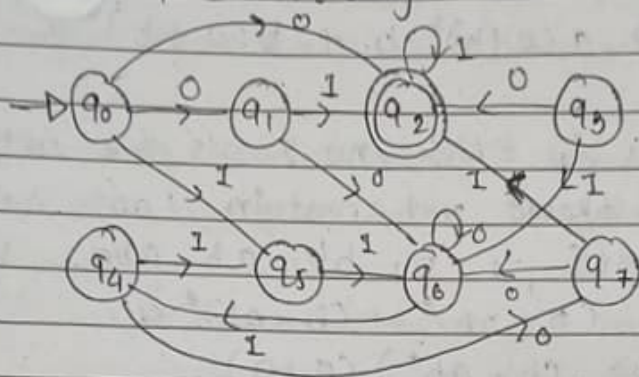
$$L = \{w \in \{a, b, c\}^* \mid \text{value of } w \text{ in base 3} \equiv 0 \pmod{4}\}$$

→ State transition table

	a	b	c
q ₀	q ₀	q ₁	q ₂
q ₁	q ₃	q ₀	q ₁
q ₂	q ₂	q ₃	q ₀
q ₃	q ₁	q ₂	q ₃



(b) Minimize the following DFA.



Step-1

→ State transition table

	0	1
q ₀	q ₁	q ₅
q ₁	q ₆	q ₂
q ₂ *	q ₀	q ₂
q ₃	q ₂	q ₆
q ₄	q ₁	q ₅
q ₅	q ₂	q ₆
q ₆	q ₆	q ₄
q ₇	q ₆	q ₂

Step-2

→ q₃ state is not reachable from initial state q₀, so we have to delete that state.

0 eq → [q₀, q₁, q₂, q₅, q₆, q₇] [q₂]

1 eq → [q₀, q₄, q₆] [q₅] [q₁, q₇] [q₂]

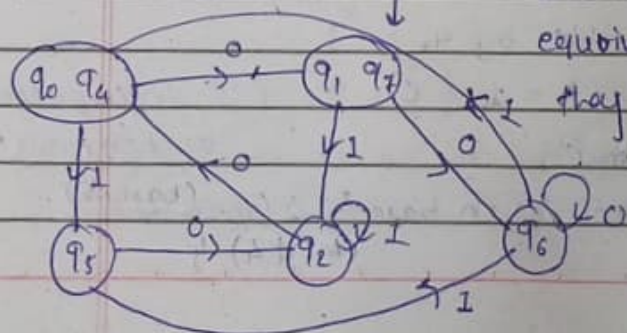
2 eq → [q₀, q₄] [q₆] [q₅] [q₁, q₇] [q₂]

3 eq → [q₀, q₄] [q₆] [q₁, q₇] [q₅] [q₂]

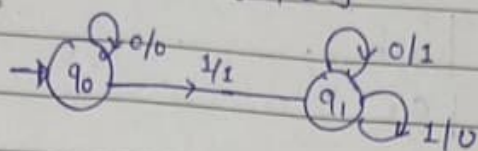
Step-3

→ For making equivalent, which states

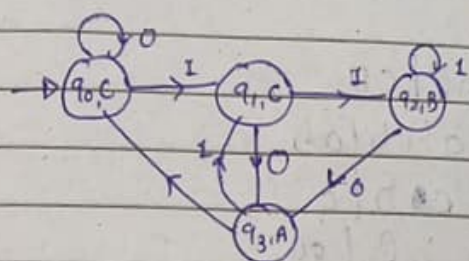
are in some group means they are equivalent, otherwise not equivalent and they are separated.



- c) Construct a Mealy's Machine that takes a binary number as input and produces 2's complement of that number as output. Assume that the string is read from LSB to MSB and each end carry is discarded. $\Sigma = \{0, 1\}$

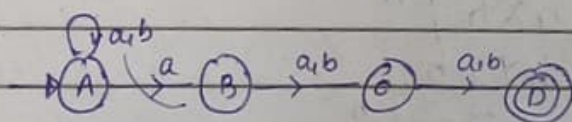


- d) Construct Moore's Machine that takes set of all string over $\Sigma = \{0, 1\}$ and produces "A" as output if string ends with "10" or produces "B" as output if string ends with "11" otherwise produces "C". $\Sigma = \{0, 1\}$

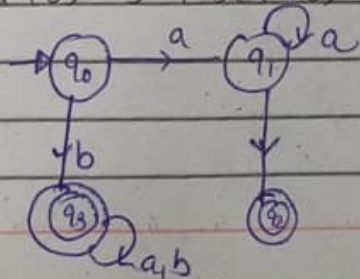


- e) Construct an NFA over $\Sigma = \{a, b\}$ for set of all strings in which 3rd symbol from the right hand side (RHS) is "a".

$$L = \{aaa, aba, \dots\}$$



- f) Convert following Regular expression to DFA. $a(a+b)^*b + b(a+b)^*a$



(10) Construct a context free grammar (CFG) for the following defined over $\Sigma = \{a, b\}$:

a) $L = \{a^i b^j c^k \mid i = j + k \text{ where } i, j, k \geq 1\}$

$$S \rightarrow XY$$

$$X \rightarrow aXb \mid ab$$

$$Y \rightarrow aYc \mid ac$$

(b) $L = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cB \mid c$$

$$S \rightarrow CD$$

$$C \rightarrow aC \mid a$$

$$D \rightarrow bDc \mid bc$$

Answer $\Rightarrow S \rightarrow AB \mid CD$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow cB \mid c$$

$$C \rightarrow aC \mid a$$

$$D \rightarrow bDc \mid bc$$

(c) $L = \{a^i b^j c^k \mid j > i + k \text{ where } i, k \geq 1\}$

$$S \rightarrow XYZ$$

$$X \rightarrow aXb \mid \wedge$$

$$Y \rightarrow bY \mid b$$

$$Z \rightarrow bZc \mid \wedge$$